Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 11 - PAO 2026 – II

Agenda

1. Reflexiones sobre el foro
2. Algoritmo de entrenamiento con GD
   2.1 Corrección del error – Backpropagation
3. El Jocabian
4. Regularización en las redes MLP

Aprendizaje: Entrenamiento con GD

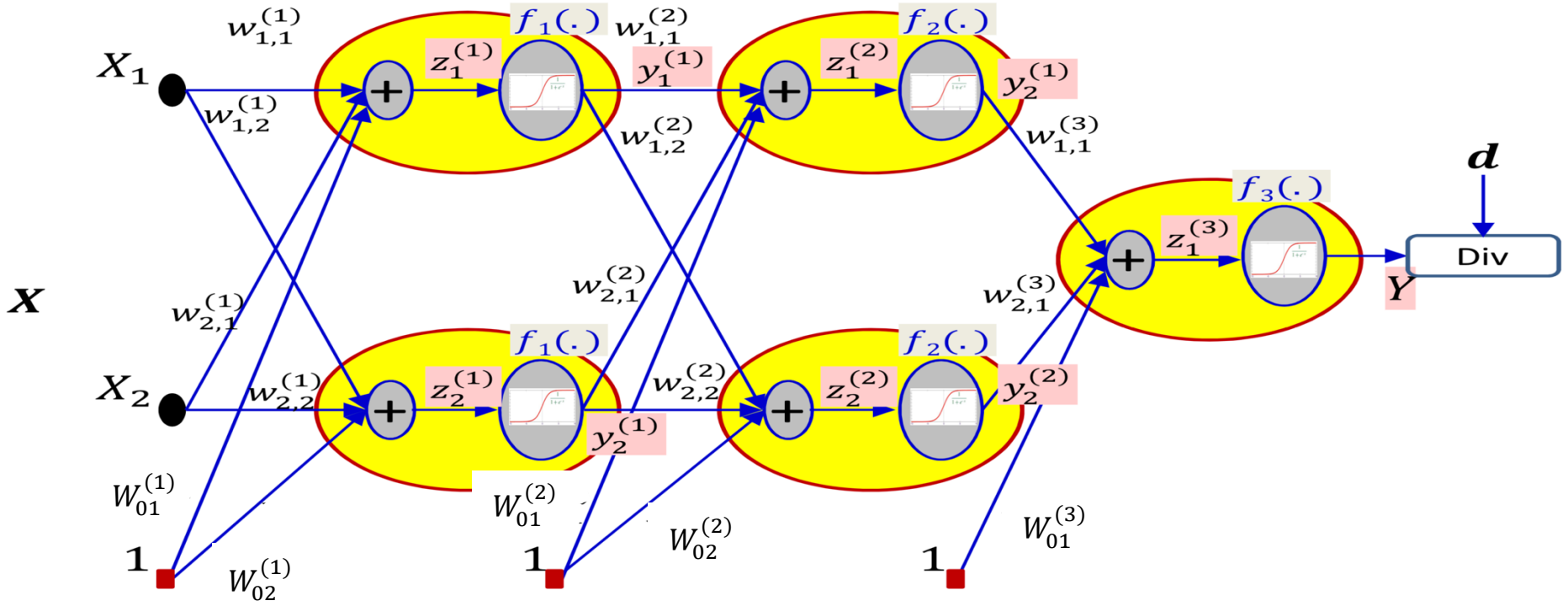
El error total en entrenamiento:

$$Error = \frac{1}{M}\sum_{m} Div(Y_m, d_m)$$

La derivada:

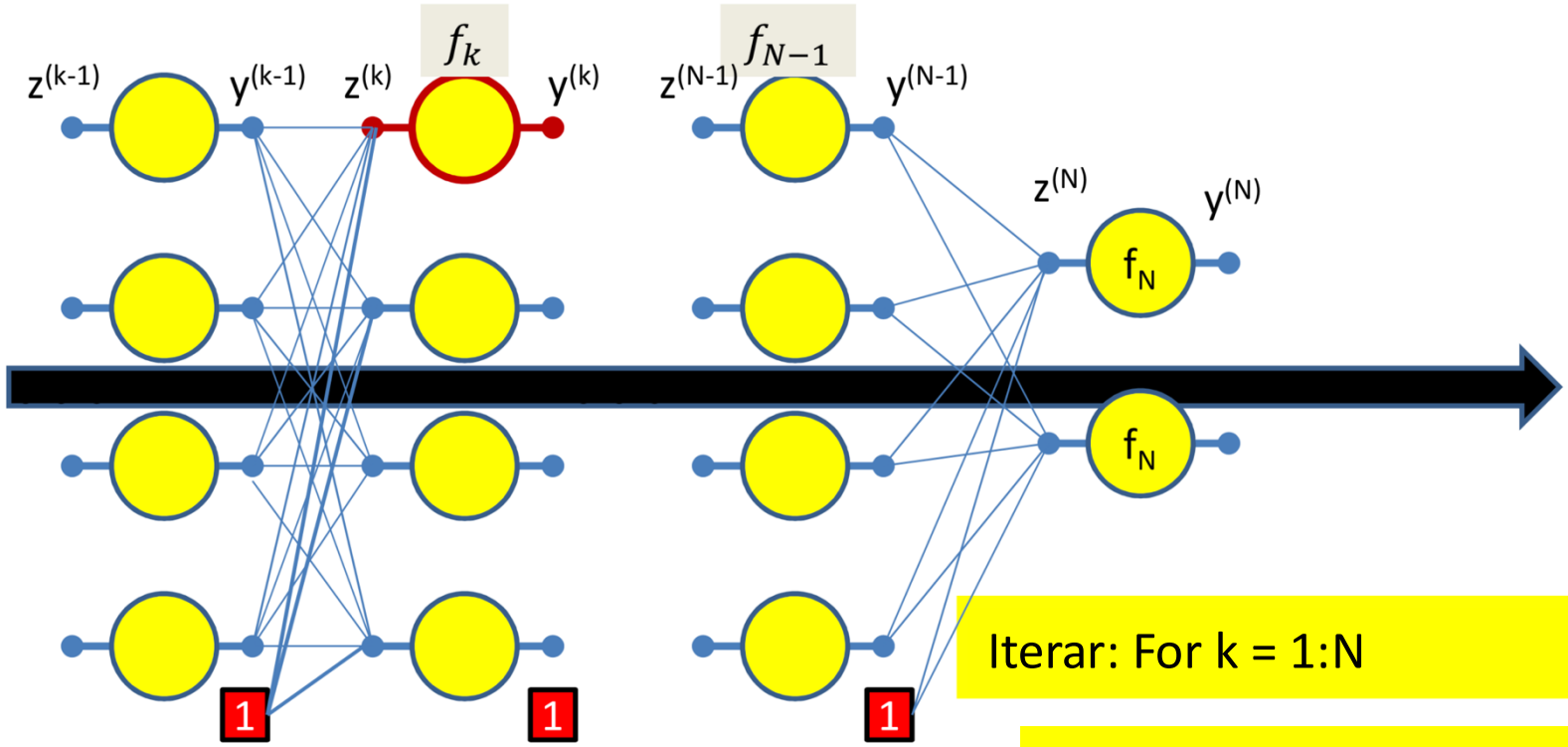
$$\frac{dError}{dW_{i,j}^{(k)}} = \frac{1}{M}\sum_{m} \frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$$

Aprendizaje:



- El propósito es calcular la derivada de $Div(Y, d)$ c.r.a. cada peso $w_{ij}^{(k)}$ y salida y_i

Aprendizaje: Forward



Iterar: For k = 1:N

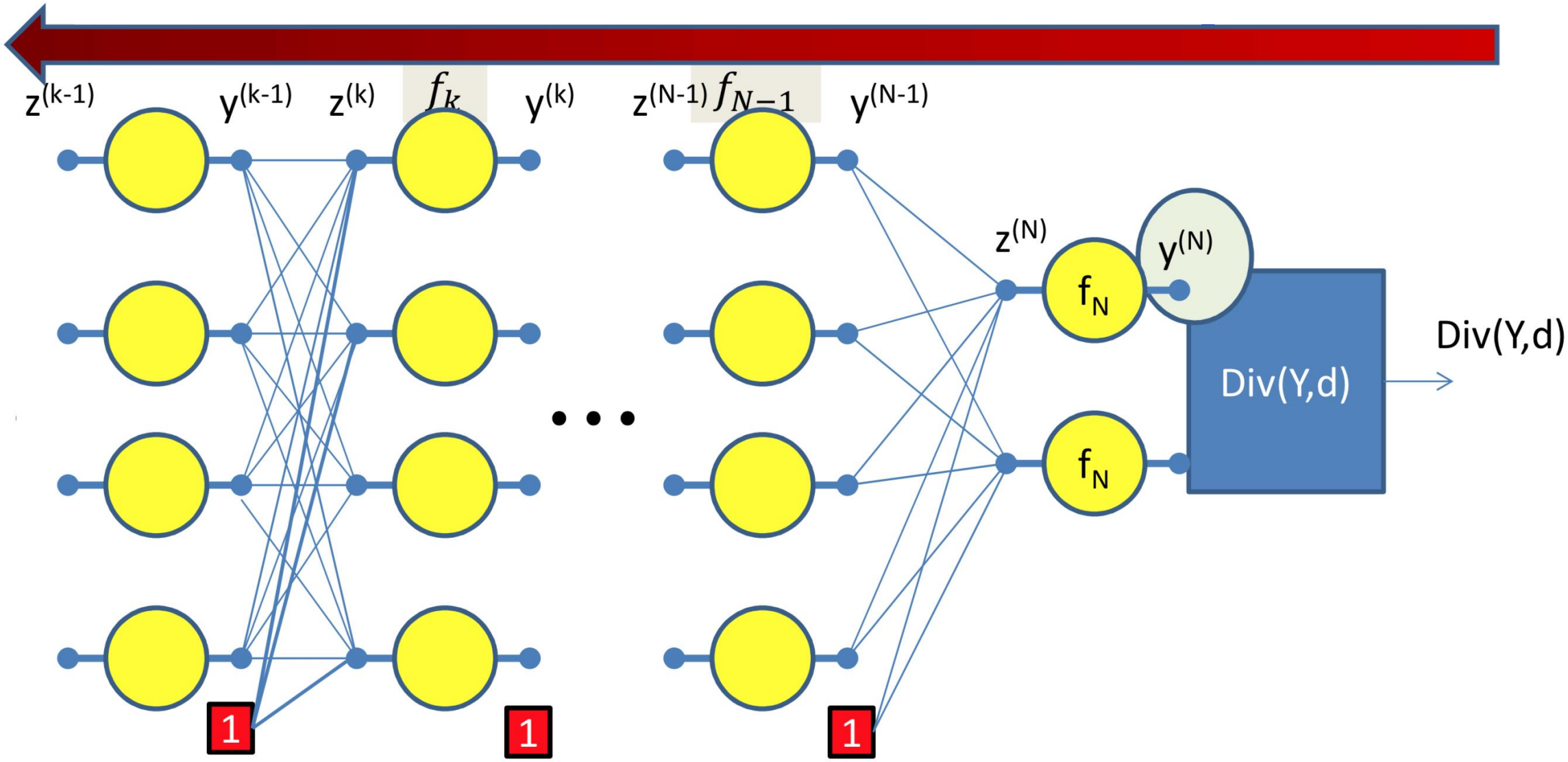
For i,j = 1:Ancho de la capa

$$y_i^{(0)} = x_i$$

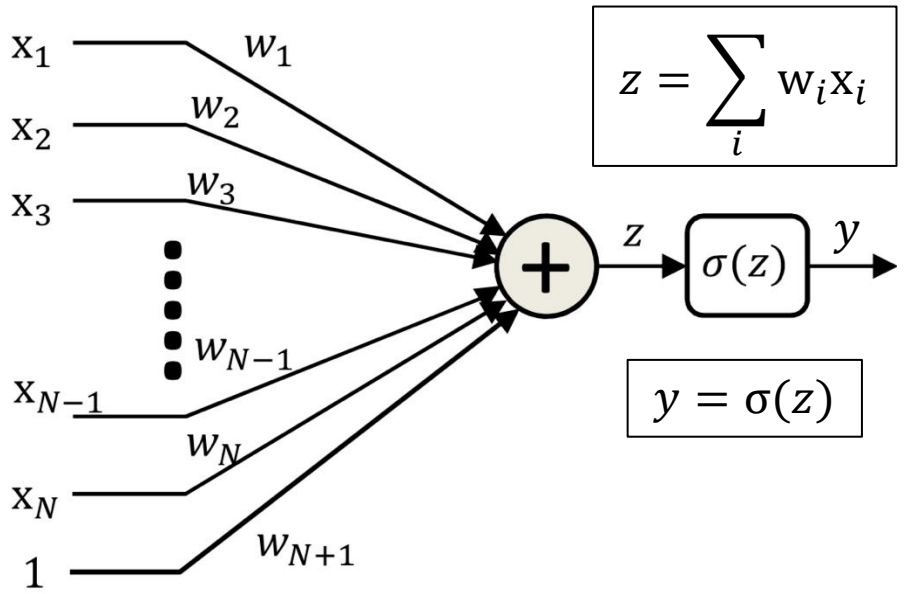
$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_i^{(k-1)}$$

$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

Aprendizaje: Backward



Aprendizaje: Perceptron con activación derivable



$$z = \sum_i w_i x_i$$

$$y = \sigma(z)$$

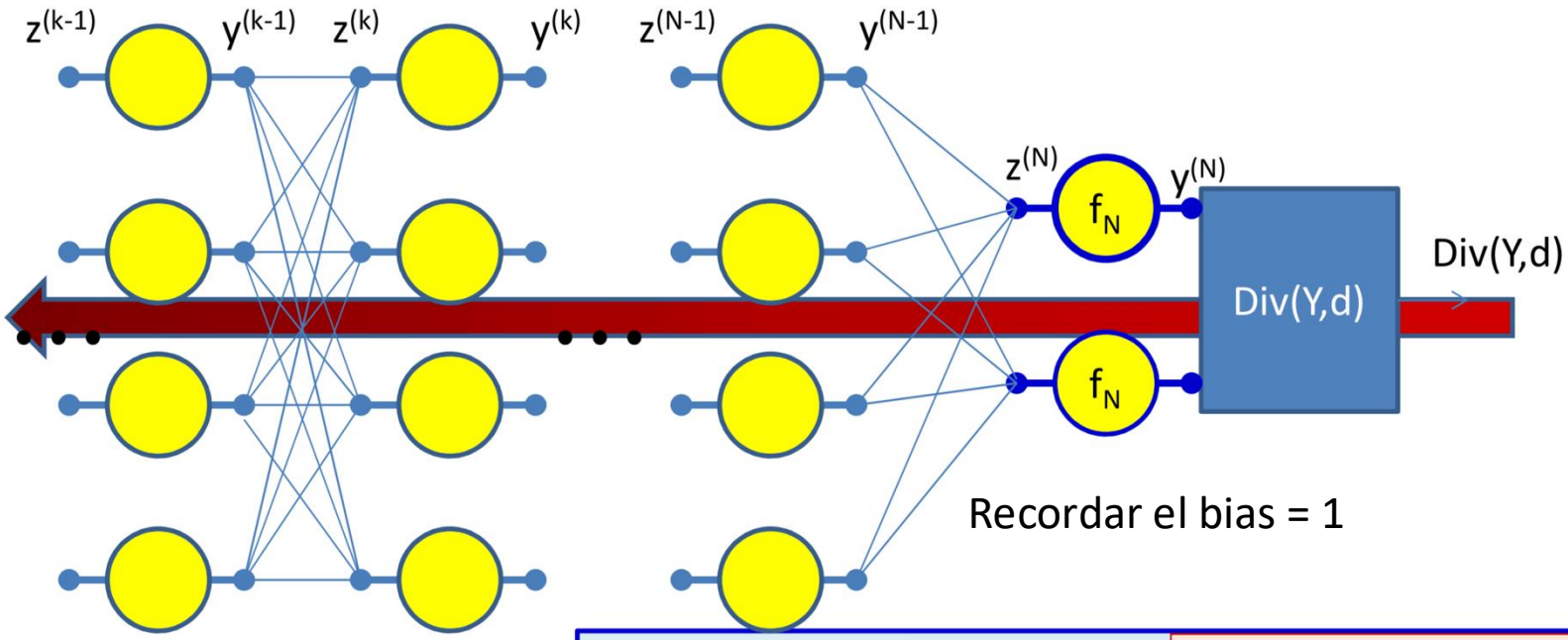
$$\frac{dy}{dw_i} = \frac{dy}{dz}\frac{dz}{dw_i}$$

$$\frac{dy}{dX_i} = \frac{dy}{dz}\frac{dz}{dX_i}$$

y_i

- Estas derivadas nos dicen cuan sensible es una función con respecto a las variables que la definen; es decir, cómo un pequeño cambio de las variables impacta al valor total de la función

Aprendizaje: Backward



Recordar el bias = 1

Inicializar: Gradiente c.r.a salida de la red

$$\frac{\partial Div}{\partial y_i} = \frac{\partial Div(Y,d)}{\partial y_i^{(N)}}$$

For k = N-1..1
For i, j = 1..Ancho de la capa

$$\frac{\partial Div}{\partial z_i^{(k)}} = f_k'\left(z_i^{(k)}\right)\frac{\partial Div}{\partial y_i^{(k)}}$$

$$\frac{\partial Div}{\partial y_i^{(k-1)}} = \sum_j w_{ij}^{(k)}\frac{\partial Div}{\partial z_j^{(k)}}$$

$$\frac{\partial Div}{\partial w_{ij}^{(k)}} = y_i^{(k-1)}\frac{\partial Div}{\partial z_j^{(k)}}$$

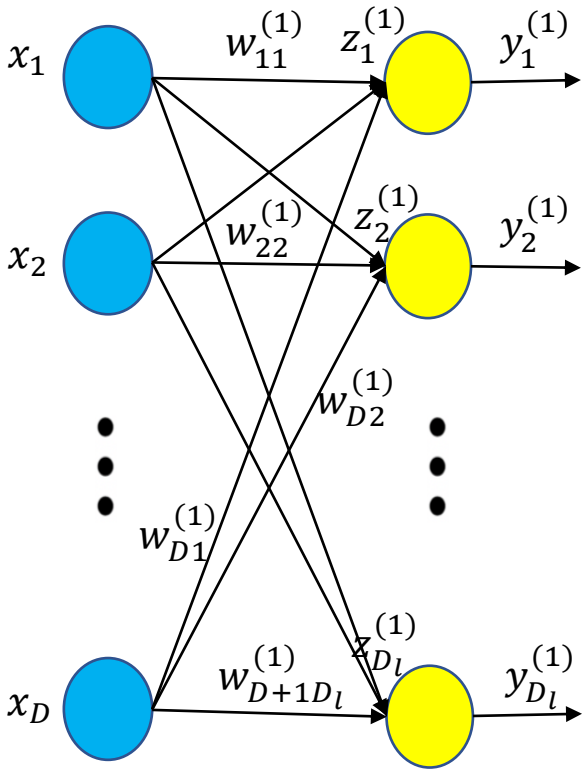
Aprendizaje: Entrenamiento con BP

- Inicializar todos los pesos $\{W^{(1)}, W^{(2)}, \ldots, W^{(K)}\}$
- Realizar:
  - Inicializar $Error = 0$, Para todas las i, j, k, inicializar $\frac{dError}{dW_{i,j}^{(k)}} = 0$
  - Para todos los ejemplos $m = 1: M$
    - Para cada capa k y para todas las unidades i, j:
      - Paso "Forward":
        - Calcular la salida Y_m
        - Calcular $Error \mathrel{+}= Div(Y_m, d_m)$
      - Paso "Backward":
        - Calcular: $\frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$
        - Calcular: $\frac{dError}{dW_{i,j}^{(k)}} \mathrel{+}= \frac{dDiv(Y_m, d_m)}{dW_{i,j}^{(k)}}$
  - Para todas las i, j, k actualizar:

$$W_{i,j}^{(k)} = W_{i,j}^{(k)} - \frac{\propto}{M} \frac{dError}{dW_{i,j}^{(k)}}$$

  - Hasta que el $Error$ converja

Aprendizaje: En Deep Learning – formulación vectorial



$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \qquad Z_k = \begin{bmatrix} z_1^{(k)} \\ z_2^{(k)} \\ \vdots \\ z_{D_k}^{(k)} \end{bmatrix} \qquad y_k = \begin{bmatrix} y_1^{(k)} \\ y_2^{(k)} \\ \vdots \\ y_{D_k}^{(k)} \end{bmatrix}$$

$$W = \begin{bmatrix} w_{01}^{(1)} & w_{01}^{(2)} & \cdots & w_{01}^{(N)} \\ w_{11}^{(1)} & w_{11}^{(2)} & \cdots & w_{11}^{(N)} \\ \vdots & \vdots & & \vdots \\ w_{D_{l-1}D_l}^{(1)} & w_{D_{l-1}D_l}^{(2)} & \cdots & w_{D_{l-1}D_l}^{(N)} \end{bmatrix} \qquad b_k = \begin{bmatrix} b_1^{(k)} \\ b_2^{(k)} \\ \vdots \\ b_{D_l}^{(k)} \end{bmatrix}$$

- El cálculo en forma vectorial de una capa se expresa:

$$Z_k = W_k y_{k-1}$$
$$y_k = f_k(Z_k)$$

Aprendizaje: Formulación vectorial

- La derivada de una función vectorial c.r.a un vector de entrada se llama "Jacobian"
- Es una matriz de derivadas parciales, que tiene la forma:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = f\left(\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_D \end{bmatrix}\right)$$

En notación vectorial

$$\mathbf{y} = f(\mathbf{z})$$

$$J_{\mathbf{y}}(\mathbf{z}) = \begin{bmatrix} \frac{\partial y_1}{\partial z_1} & \frac{\partial y_1}{\partial z_2} & \cdots & \frac{\partial y_1}{\partial z_D} \\ \frac{\partial y_2}{\partial z_1} & \frac{\partial y_2}{\partial z_2} & \cdots & \frac{\partial y_2}{\partial z_D} \\ \cdots & \cdots & \ddots & \cdots \\ \frac{\partial y_M}{\partial z_1} & \frac{\partial y_M}{\partial z_2} & \cdots & \frac{\partial y_M}{\partial z_D} \end{bmatrix}$$

$$\Delta y = J_y(z)\Delta z$$

Regularización MLP: Dropout para manejar overfitting

Dropout

- A cada unidad de procesamiento de la RN (excepto las que están en la capa de salida) se le da una probabilidad *p* de ser temporalmente ignorada en los cálculos.
- El hiper-parámetro *p* se denomina tasa de dropout.
- En cada iteración, se selecciona aleatoriamente las neuronas que se dejan fuera (dropout), de acuerdo con la probabilidad asignada.
- Como resultado, cada vez trabajamos con una RN más pequeña.
- El dropout ignora unidades de procesamiento durante la fase de entrenamiento. Algunas unidades no se consideran durante el paso forward ni en backward.